Name:Krishna Dalvi

D15C/22

Experiment – 8

AIM

To design, train, and evaluate a Convolutional Neural Network (CNN) on the **CIFAR-10 dataset** for image classification.

---

1. Dataset Source

Dataset: CIFAR-10 Image Dataset
Platform: TensorFlow / Keras

The CIFAR-10 dataset contains **colored images** belonging to 10 different classes such as airplane, car, bird, cat, etc.

Dataset contains:

- **60,000 images total**
- **50,000 training images**
- **10,000 testing images**
- **Each image size: 32 × 32 pixels**
- **3 color channels (RGB)**

---

2. Dataset Overview

Feature	Description
Image Size	32 × 32 pixels
Image Type	Colored (RGB)
Training Samples	50,000
Testing Samples	10,000
Classes	10 (Airplane, Car, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck)

---

3. CNN Architecture

Layer	Description
Conv2D	32 filters, 3×3, ReLU
MaxPooling2D	2×2 pooling
Conv2D	64 filters, 3×3, ReLU
MaxPooling2D	2×2 pooling
Flatten	Converts to 1D
Dense	64 neurons, ReLU
Output Dense	10 neurons, Softmax

---

4. Methodology / Procedure

Step 1: Import Libraries

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models
```

Step 2: Load Dataset

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Step 3: Normalize Pixel Values

```
X_train, X_test = X_train / 255.0, X_test / 255.0
```

Step 4: Verify Shape

```
print(X_train.shape)
print(X_test.shape)
```

Output:

```
(50000, 32, 32, 3)
(10000, 32, 32, 3)
```

Step 5: Construct CNN Model

```
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Step 6: Compile Model

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Step 7: Train Model

```
model.fit(X_train, y_train, epochs=5)
```

Step 8: Evaluate Model

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
```

Step 9: Make Predictions

```
class_names = ['Airplane','Car','Bird','Cat','Deer',
               'Dog','Frog','Horse','Ship','Truck']

predictions = model.predict(X_test[:5])

for i in range(5):
    plt.imshow(X_test[i])
    plt.title(f"Predicted: {class_names[predictions[i].argmax()]}, Actual:
{class_names[y_test[i][0]]}")
    plt.axis('off')
    plt.show()
```

---

5. Code (Visualization)

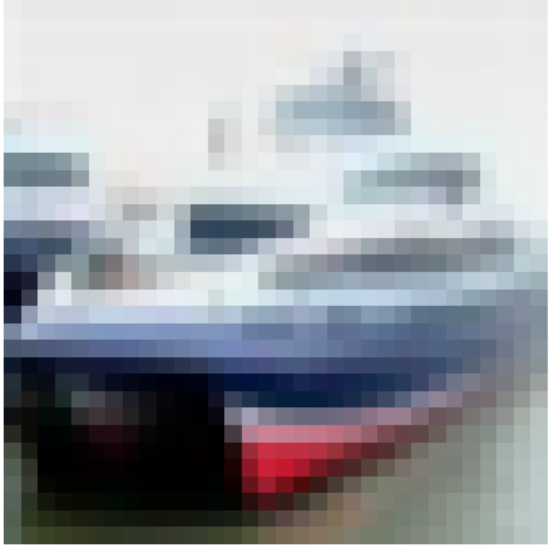
```
plt.figure(figsize=(8,8))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(X_train[i])
    plt.title(y_train[i][0])
    plt.axis('off')
plt.show()
```

---

6. Output / Screenshots

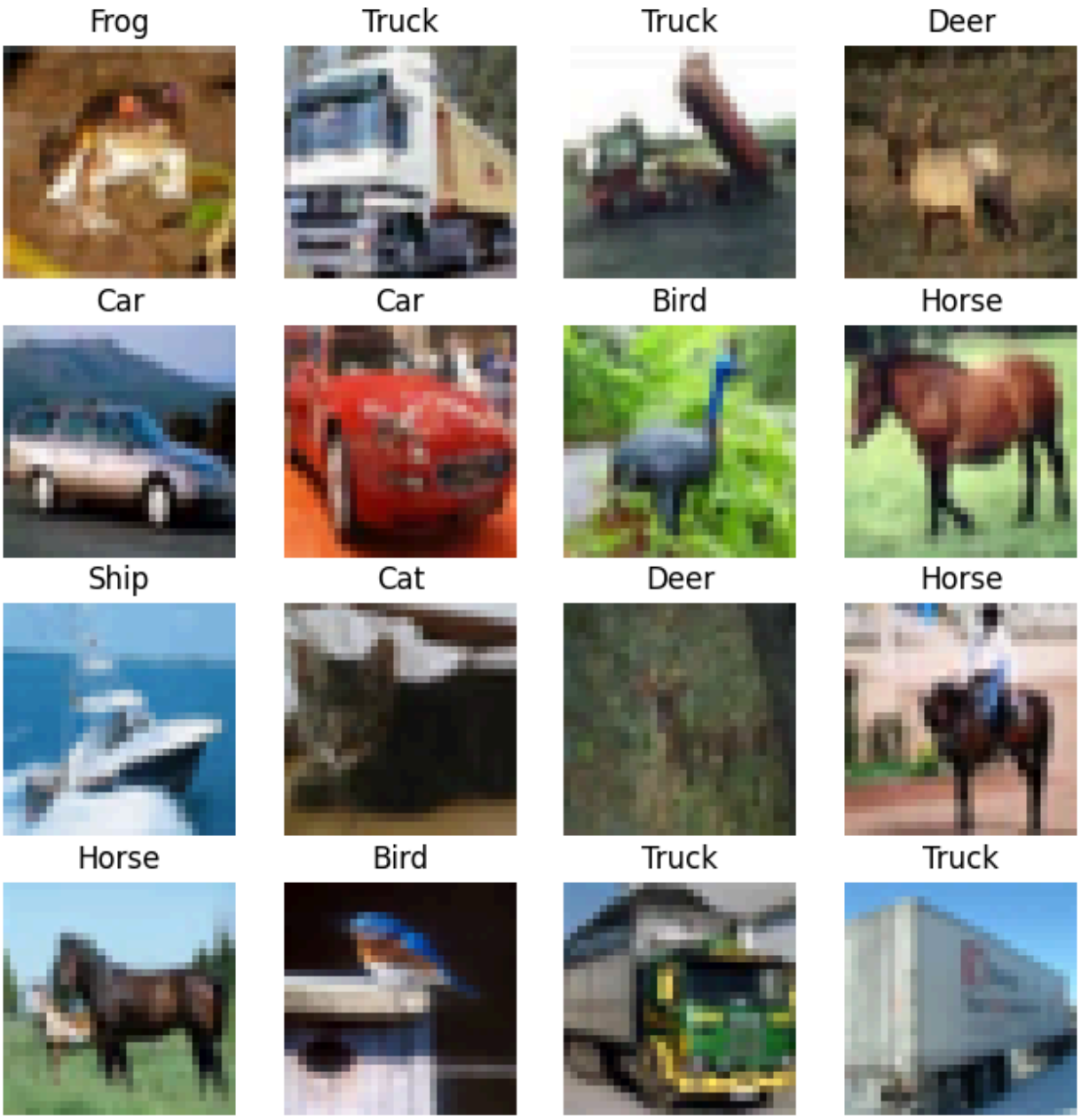
Predicted: Cat, Actual: Cat

Predicted: Ship, Actual: Ship



Predicted: Airplane, Actual: Airplane

Predicted: Deer, Actual: Frog



---

7. Observations

- CNN achieved around **70–75% accuracy** on CIFAR-10.
- Model successfully classified different object categories.
- Feature extraction is more complex compared to MNIST due to color images.
- Some confusion occurs between similar classes (e.g., cat vs dog).

---

8. Limitations

- Lower accuracy compared to MNIST due to dataset complexity
- Requires more epochs and deeper networks for better performance
- Computational cost is higher for RGB images
- May require data augmentation for improved results

---

9. Conclusion

In this experiment, a CNN was successfully trained on the **CIFAR-10 dataset**. The model demonstrated the ability to classify real-world colored images.

This experiment highlights:

- CNN effectiveness in **image classification tasks**
- Importance of **feature extraction in complex datasets**
- Need for **deeper architectures and tuning** for better accuracy